

VULCAN: *Instance-Optimal* Systems Heuristics Through LLM-Driven Search

Rohit Dwivedula, Divyanshu Saxena, Sujay Yadalam, Daehyeok Kim, Aditya Akella
The University of Texas at Austin

Presented By: Nitin Kedia

A history of man-made heuristics

Cache eviction. FIFO, LRU, LFU, GDSF, LIRS, ARC, TinyLFU, SIEVE, S3-FIFO

Congestion Control. Tahoe, Reno, NewReno, Vegas, CUBIC, BBR.

LLM Inference. FT, Orca, vLLM, Sarathi-Serve, DistServe.

... and many more

Require new
heuristics!

New Workload

New Objectives

New Hardware

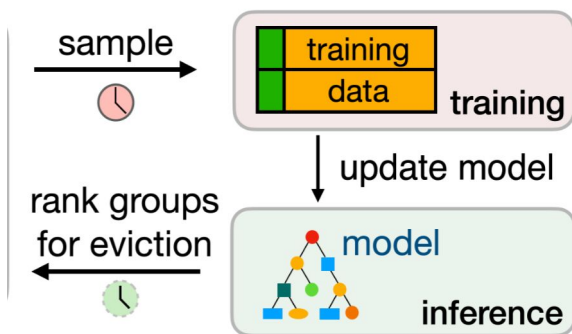
New Constraints



ML to the rescue?

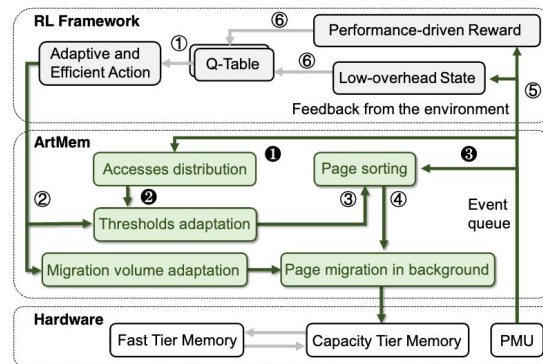
CDN caching

HALP, LRB, GLCache, LHD



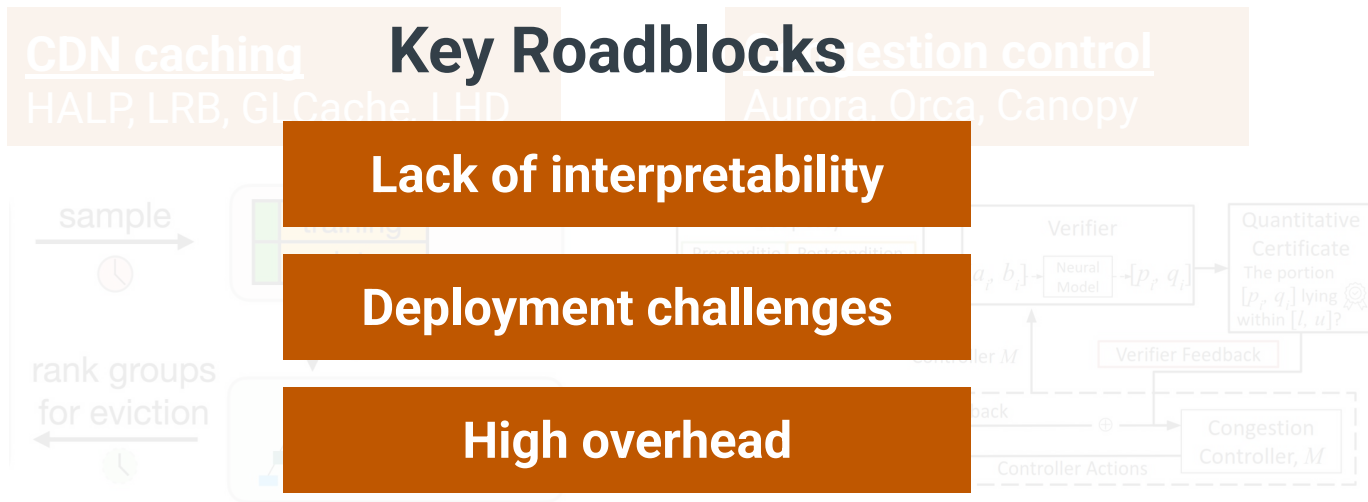
Memory Tiering

ArtMem, MEMTIS, ARMS



... and many more.

ML to the rescue?



... and many more.

What is instance-optimality? Why is it needed?

Data: CloudPhysics dataset (106 traces from web servers)

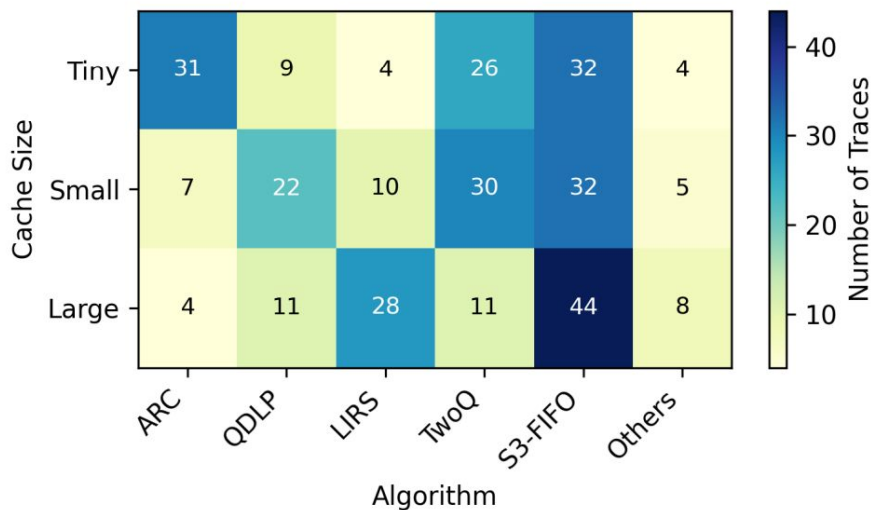


Figure: Count of CloudPhysics traces where each heuristic has highest object hit rate.

Vulcan: Overview

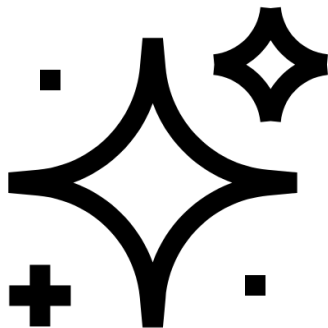


Vulcan



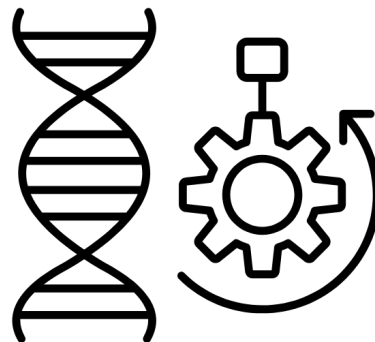
Vulcan

LLMs



+

Evolutionary search



Vulcan: End-to-End Overview

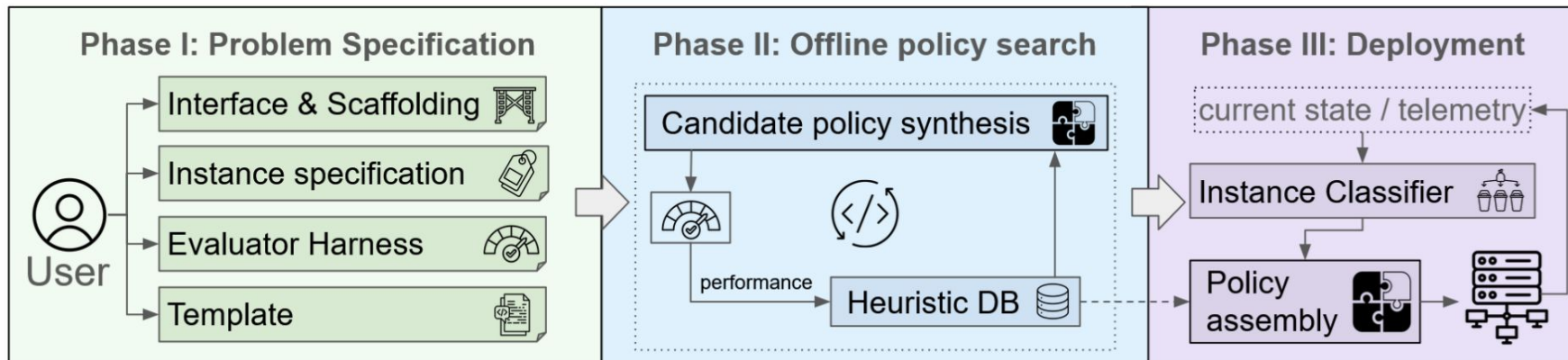
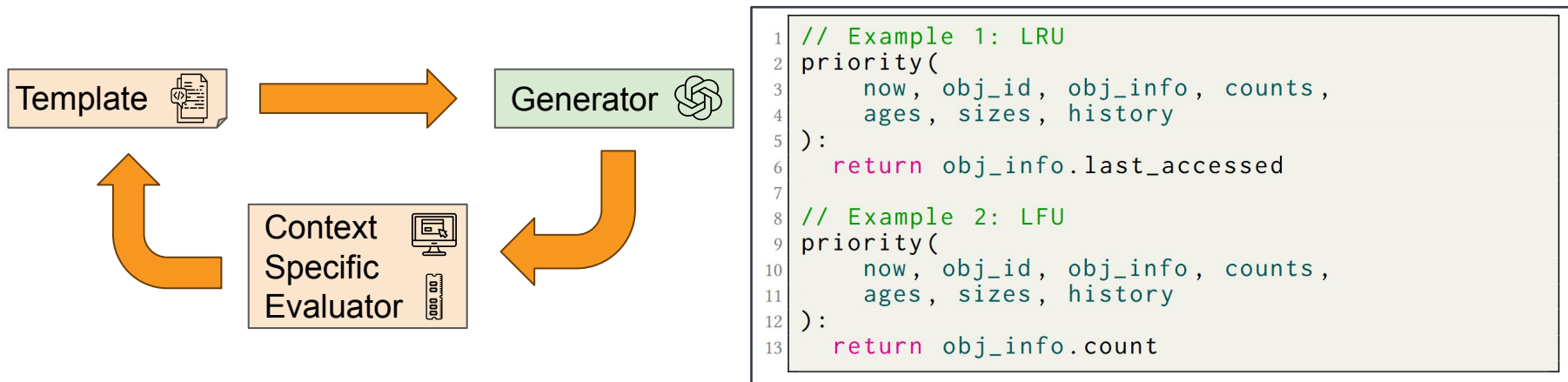


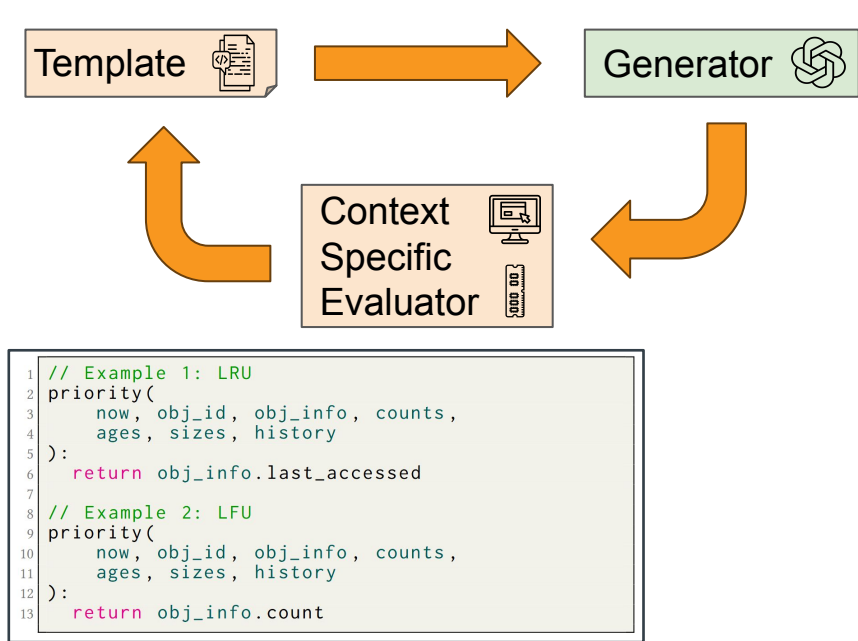
Figure 3: **End-to-end overview of VULCAN.** Each of the inputs defined in Phase I are used by subsequent phases.



Providing Example Heuristics as a Starting Point



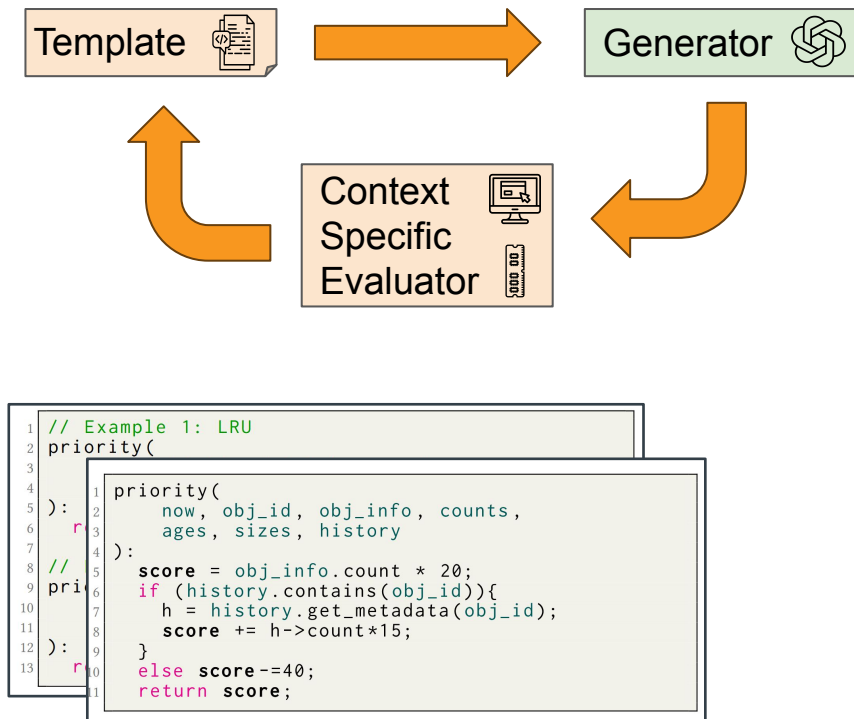
After a Couple of Iterations ...



```
1 priority(
2     now, obj_id, obj_info, counts,
3     ages, sizes, history
4 ):
5     score = obj_info.count * 20;
6     if (history.contains(obj_id)){
7         h = history.get_metadata(obj_id);
8         score += h->count*15;
9     }
10    else score -=40;
11    return score;
```



After More Iterations ...



```
1 priority(
2     now, obj_id, obj_info, counts,
3     ages, sizes, history
4 ):
5     score =
6     age =
7     score = age,
8     score -= obj_info.size/500;
9     if (history.contains(obj_id)){
10         h = history.get_metadata(obj_id);
11         score += h->count*15;
12         score += h->age_at_eviction_time/150;
13     }
14     else score -=40;
15     recent = ages.percentile(0.75);
16     if (obj_info.last_accessed<recent) score -=30;
17     big = sizes.percentile(0.75);
18     if (obj_info.size > big) score -=25;
19     else score +=10;
20     frequent = counts.percentile(0.7);
21     score += (obj_info.count>frequent) ? 50:-5;
22     if (age < 100) score +=25;
23     if (obj_info.count < 2) score -= 15;
24     r
```

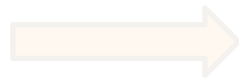
Recently evicted objects prioritized to be kept in cache.

Popular objects (top 30%ile) are prioritized.



Key Advantages

Template



Generator



Interpretable

Easily deployable

Minimal overhead

```
1 // Example 1: LRU
2 priority(
3     now, obj_id, obj_info, counts,
4     ages, sizes, history
5 ):
6     score = obj_info.count * 20;
7     if (history.contains(obj_id)){
8         h = history.get_metadata(obj_id);
9         score += h->count*15;
10    }
11    else score -= 40;
12    return score;
```

```
1 priority(
2     now, obj_id, obj_info, counts,
3     ages, sizes, history
4 ):
5     score =
6     age =
7     score = age;
8     score -= obj_info.size/500;
9     if (history.contains(obj_id)){
10         h = history.get_metadata(obj_id);
11         score += h->count*15;
12         score += h->age_at_eviction_time/150;
13     }
14     else score -= 40;
15     recent = ages.percentile(0.75);
16     if (obj_info.last_accessed < recent) score -= 30;
17     big = sizes.percentile(0.75);
18     if (obj_info.size > big) score -= 25;
19     else score += 10;
20     frequent = counts.percentile(0.7);
21     score += (obj_info.count > frequent) ? 50 : -5;
22     if (age < 100) score += 25;
23     if (obj_info.count < 2) score -= 15;
24     return score;
```

Recently evicted objects prioritized to be kept in cache.

Popular objects (top 30%ile) are prioritized.

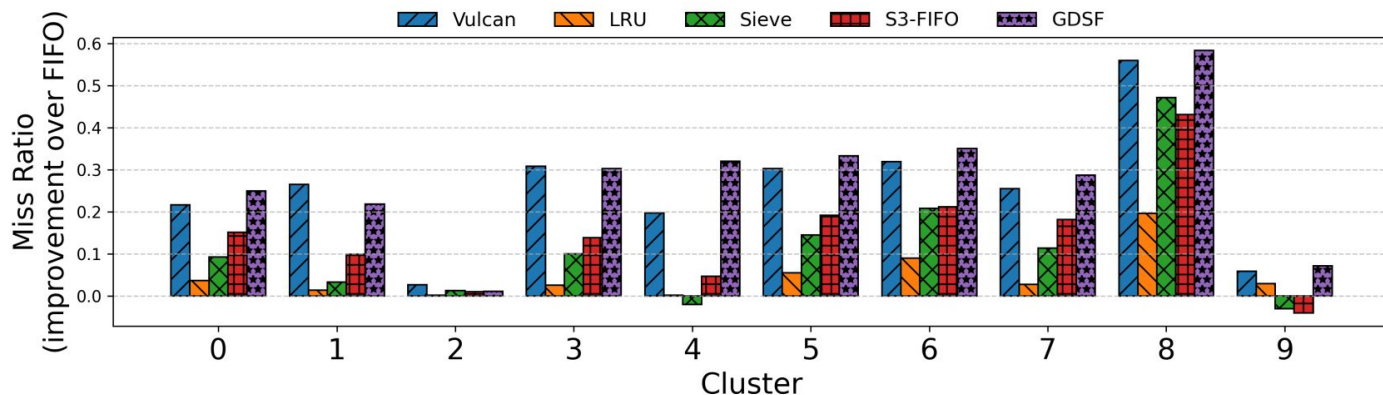
How does Vulcan perform?



Baselines: 13 heuristics spanning decades of research.

GDSF, S3-FIFO, SIEVE, LIRS, LHD, Cacheus, FIFO-Reinsertion, LeCaR, SR-LFU, CR-LRU, LRU, MRU, and FIFO.

Data: CloudPhysics dataset (106 traces from web servers)



- Best performing in 3 clusters (C1 – 21%, C2 – 69%, C3 – 1.9%)
- Second best in 4 clusters (C5, C6, C7, C9), Third everywhere else.
- Total OpenAI API cost to find all these heuristics: < 100\$

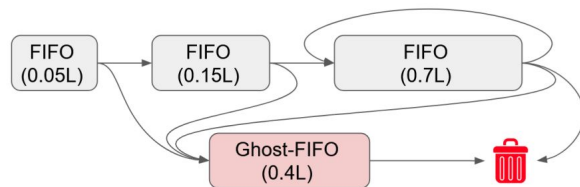
Importance of Input Specification in Vulcan



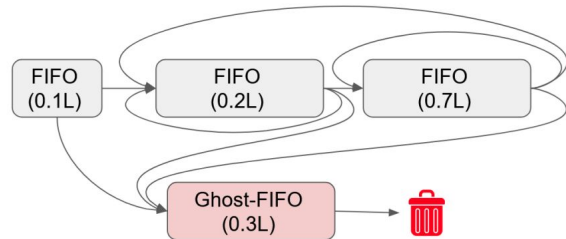
Queue based Cache Eviction: Vulcan should only search this kind.

Input Specification:

1. Queue Config
2. Per-Object Features
3. Initial Placement Function
4. Transition Function



(a) Topology discovered for C_7



(b) Topology discovered for C_8

Figure 9: Two cache queue topologies discovered by VULCAN. Each

Why could this work?

- **Brute force:** The ability to test thousands (if not millions) of candidate heuristics with minimal human effort
- **Incremental changes:** Newer heuristics are more often remixes and improvements over the status quo than radically new designs.
 - e.g., FIFO → CLOCK → CLOCK-Pro (caching)
 - e.g., FireHose 1.0 → FH 1.1 → WatchTower (or) Saturn → Jupiter (network topology @ Google)
- **LLMs have domain knowledge.** Researchers use their intuition, across all systems problems, to build these tweaks. Pre-training might be encoding this broad knowledge base.

Summary

Instance-optimality is essential, and Vulcan offers a practical way of achieving it

Exciting results for caching and memory tiering

How do you know you're in a different "instance"?

How do you draw the line between what is specified and what is learnt?

My thoughts

- **No mechanism, only policy:** In both the case studies Vulcan uses mechanisms, structures (eg. Queue based Caching Techniques) discovered by decades of research. Only the final fine-tuning is done by the machine.
- **Runtime switching of instance-optimal policies:** There is no end-to-end result of switching between optimal policies in a E2E system.
- **How is different from AlphaEvolve?**

How does Vulcan perform?



Baselines: 13 heuristics spanning decades of research.

GDSF, S3-FIFO, SIEVE, LIRS, LHD, Cacheus, FIFO-Reinsertion, LeCaR, SR-LFU, CR-LRU, LRU, MRU, and FIFO.

Data: CloudPhysics dataset (106 traces from web servers)

Trace w89



A

Dataset	%age of traces	
CloudPhysics	$\mathcal{A}$ (48%)	$\mathcal{B}$ (42%)
	$\mathcal{C}$ (14%)	$\mathcal{D}$ (31%)
MSR	$\mathcal{W}$ (57%)	$\mathcal{X}$ (64%)
	$\mathcal{Y}$ (57%)	$\mathcal{Z}$ (21%)

- 2.8% better miss ratio than best performing baseline
- Outperforms all baselines on 51 / 105 traces (48%)
- Total OpenAI API cost to find all these heuristics: ~\$7

How does Vulcan perform in Memory Tiering?

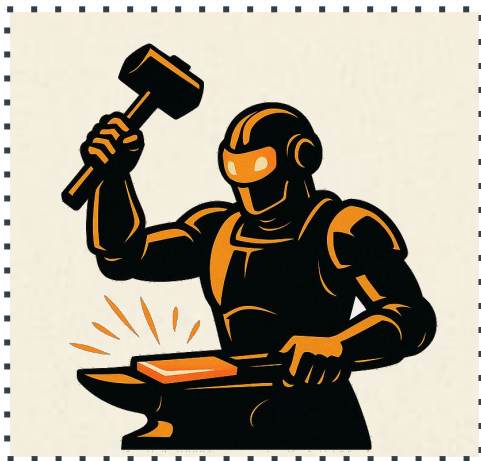
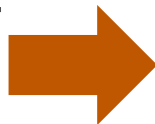
Baselines: 13 heuristics spanning decades of research.

GDSF, S3-FIFO, SIEVE, LIRS, LHD, Cacheus, FIFO-Reinsertion, LeCaR, SR-LFU, CR-LRU, LRU, MRU, and FIFO.

Data: CloudPhysics dataset (106 traces from web servers)

Possible Alternative: LLM-Driven Heuristic Search

Deployment context



LLM based code generator



Heuristic



Example: Specification for Cache Eviction Policy



Specification

Template: Description of Heuristic

Write a function that takes a list of objects in cache and decides which one(s) to evict

Template 

Examples 

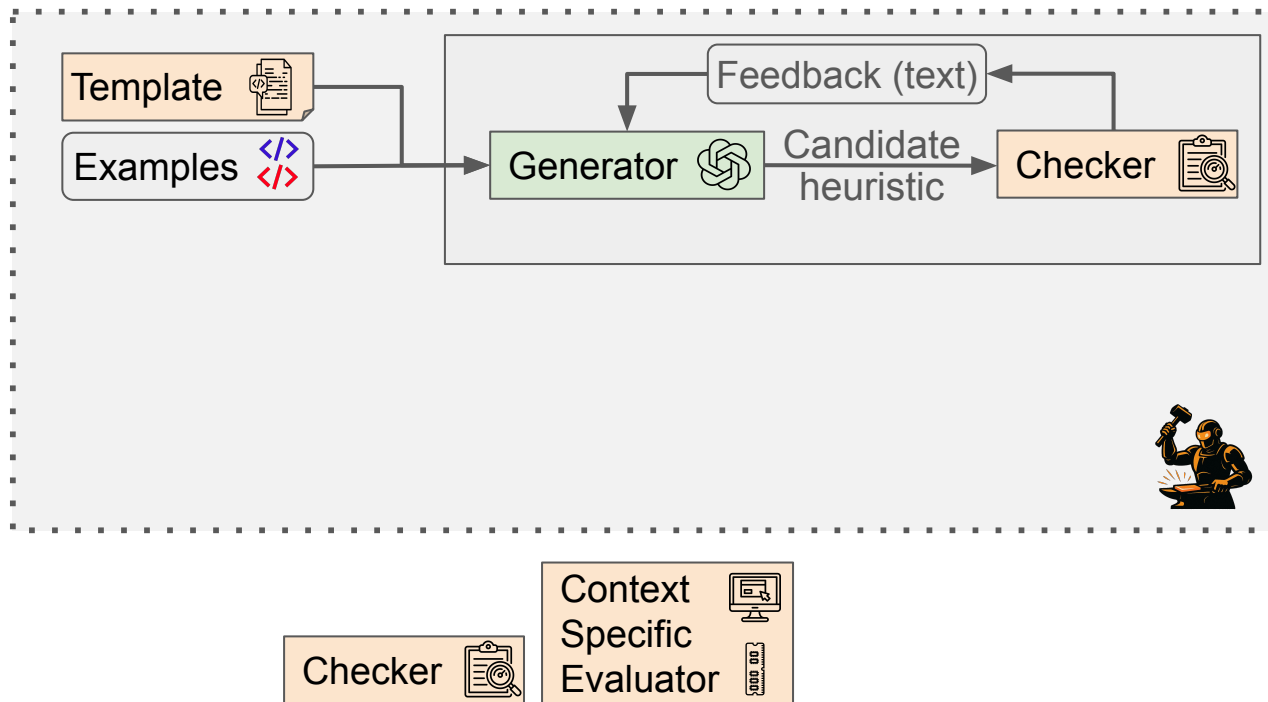


Checker 

Context
Specific
Evaluator 

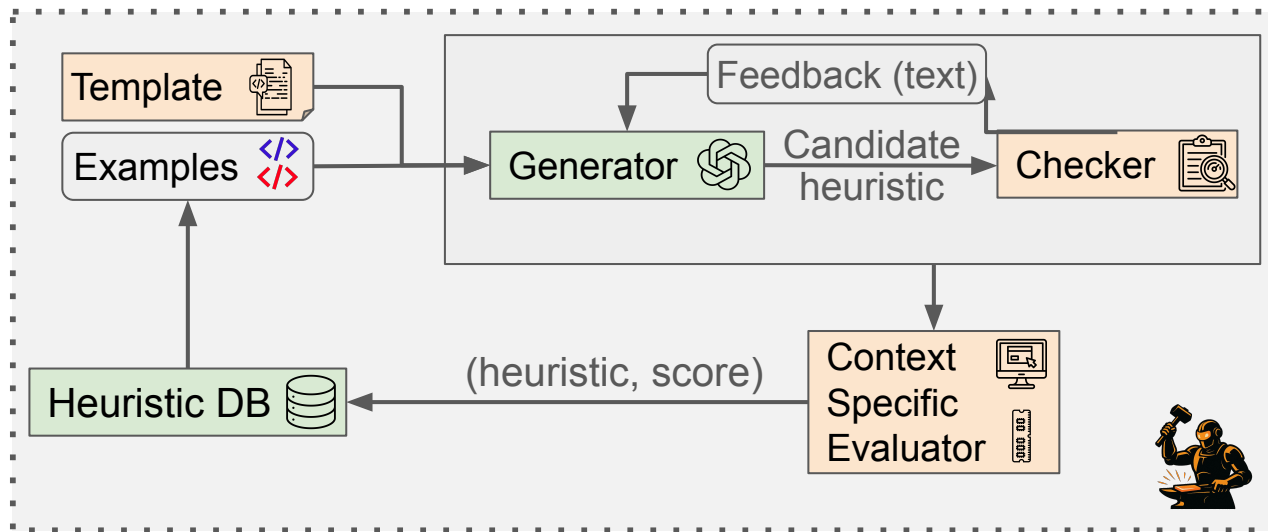
Checker: Checking the Validity of Candidate Heuristic

The function you write must be valid C++ and must compile



Evaluator: Evaluating the Goodness of Candidate Heuristic

I want high object hit rates





Workflow for Generating Cache Eviction Policies

